SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Course: Fundamentals of Data Science

Course Code: DSA0404

Slot: A

Faculty: KRISHNARAJ M

Submitted By

V. Ravi Kumar Reddy (192424304)

Problem Solving Using Numpy, Pandas, Scipy, Scikit Learn Libraries in Python for Data Scientist

1. GENERATE RAW MOCK DATA (Simulating an imperfect CSV file)

Code:

File Edit Format Run Options Window Help

```
import pandas as pd
import numpy as np

# 1. GENERATE RAW MOCK DATA (Simulating an imperfect CSV file)
raw_data = {
    "Employee_Name": [" Alice ", "Bob", "Charlie", "Diana", "Evan"],
    "Department": ["HR", "Engineering", "HR", "Engineering", "Marketing"],
    "Salary": [50000, 85000, np.nan, 92000, 60000],  # Missing value
    "Join_Date": [
        "2022-01-15",
        "2021-06-20",
        "2023-03-11",
        "2020-11-01",
        "2024-02-28",
    ],
}

# Create DataFrame
df = pd.DataFrame(raw_data)

# Display Raw Data
print("--- Raw Data ---")
print(df)
```

Output:

```
--- Raw Data ---
  Employee_Name   Department   Salary   Join_Date
0         Alice           HR  50000.0  2022-01-15
1           Bob  Engineering  85000.0  2021-06-20
2       Charlie           HR      NaN  2023-03-11
3         Diana  Engineering  92000.0  2020-11-01
4          Evan    Marketing  60000.0  2024-02-28
>>>
```

2. DATA PREPROCESSING

Code:

```
File  Edit  Format  Run  Options  Window  Help
import pandas as pd
import numpy as np

# Employee Data
raw_data = {
    "Employee_Name": [" Alice ", "Bob", "Charlie", "Diana", "Evan"],
    "Department": ["HR", "Engineering", "HR", "Engineering", "Marketing"],
    "Salary": [50000, 85000, np.nan, 92000, 60000],
    "Join_Date": [
        "2022-01-15",
        "2021-06-20",
        "2023-03-11",
        "2020-11-01",
        "2024-02-28"
    ]
}

df = pd.DataFrame(raw_data)

# 2. DATA PREPROCESSING

# Remove extra spaces from Employee_Name
df["Employee_Name"] = df["Employee_Name"].str.strip()

# Replace missing salary with the median salary
median_salary = df["Salary"].median()
df["Salary"] = df["Salary"].fillna(median_salary)

# Convert Join_Date to datetime format
df["Join_Date"] = pd.to_datetime(df["Join_Date"])

# Create Years_of_Service column
df["Years_of_Service"] = 2026 - df["Join_Date"].dt.year

print("--- Preprocessed Data ---")
print(df)
```

Output:

```
--- Preprocessed Data ---
  Employee_Name   Department   Salary  Join_Date  Years_of_Service
0         Alice           HR  50000.0 2022-01-15                 4
1           Bob  Engineering  85000.0 2021-06-20                 5
2       Charlie           HR  72500.0 2023-03-11                 3
3         Diana  Engineering  92000.0 2020-11-01                 6
4          Evan    Marketing  60000.0 2024-02-28                 2
>>>
```

3. DATA AGGREGATION

Code:

```
File  Edit  Format  Run  Options  Window  Help
import pandas as pd
import numpy as np

# Employee Data
raw_data = {
    "Employee_Name": ["Alice", "Bob", "Charlie", "Diana", "Evan"],
    "Department": ["HR", "Engineering", "HR", "Engineering", "Marketing"],
    "Salary": [50000, 85000, 72500, 92000, 60000],
    "Join_Date": [
        "2022-01-15",
        "2021-06-20",
        "2023-03-11",
        "2020-11-01",
        "2024-02-28"
    ]
}

df = pd.DataFrame(raw_data)

# 3. DATA AGGREGATION
# Group by Department and calculate average salary and total employees
dept_summary = (
    df.groupby("Department")
      .agg(
          Avg_Salary=("Salary", "mean"),
          Total_Employees=("Employee_Name", "count")
      )
      .reset_index()
)

print("--- Aggregated Summary ---")
print(dept_summary)
```

Output:

```
--- Preprocessed Data ---
  Employee_Name   Department    Salary  Join_Date  Years_of_Service
0         Alice           HR  50000.0 2022-01-15                 4
1           Bob  Engineering  85000.0 2021-06-20                 5
2       Charlie           HR  72500.0 2023-03-11                 3
3         Diana  Engineering  92000.0 2020-11-01                 6
4          Evan    Marketing  60000.0 2024-02-28                 2
>>>
```

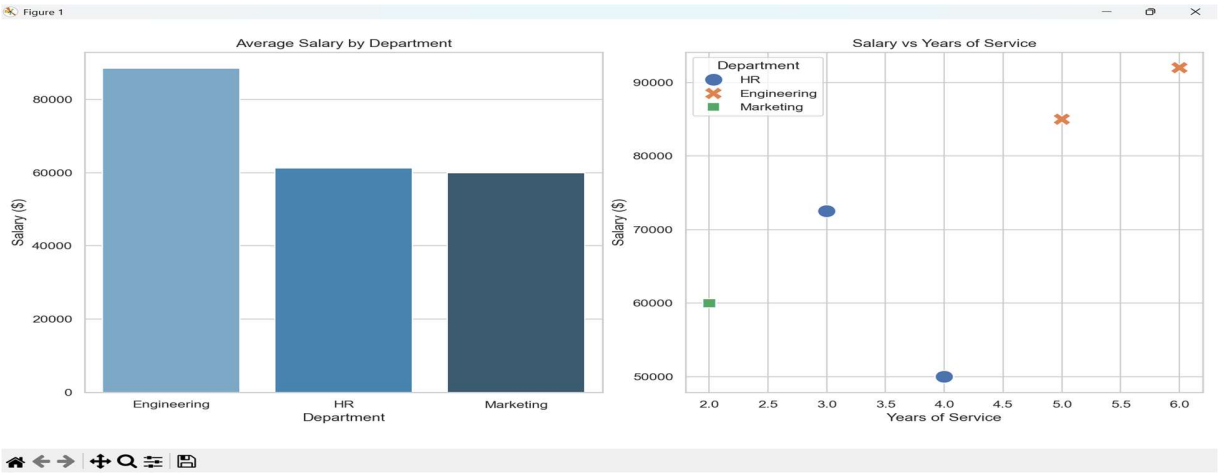
4. DATA VISUALIZATION

Code:

File Edit Format Run Options Window Help

```
        )
        .reset_index()
)

# 4. DATA VISUALIZATION

# Set theme
sns.set_theme(style="whitegrid")

# Create two plots
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: Bar Chart
sns.barplot(
    data=dept_summary,
    x="Department",
    y="Avg_Salary",
    hue="Department",
    palette="Blues_d",
    legend=False,
    ax=axes[0]
)

axes[0].set_title("Average Salary by Department")
axes[0].set_xlabel("Department")
axes[0].set_ylabel("Salary ($)")

# Plot 2: Scatter Plot
sns.scatterplot(
    data=df,
    x="Years_of_Service",
    y="Salary",
    hue="Department",
    style="Department",
    s=200,
    ax=axes[1]
)

axes[1].set_title("Salary vs Years of Service")
axes[1].set_xlabel("Years of Service")
axes[1].set_ylabel("Salary ($)")

plt.tight_layout()
plt.show()
```

Output:



Creating Sample Data Frame

Code:

```
File  Edit  Format  Run  Options  Window  Help
import pandas as pd

# Create an empty DataFrame
df = pd.DataFrame()

# Add columns
df['Name'] = ['John', 'Emma', 'Liam', 'Olivia']
df['Age'] = [20, 19, 21, 18]
df['Student'] = [True, True, False, True]

# Display the DataFrame
print(df)
```

Output:

```
    Name  Age  Student
0   John   20     True
1   Emma   19     True
2   Liam   21    False
3  Olivia  18     True
>>>
```

Adding Row to Data Frame

Code:

```
File  Edit  Format  Run  Options  Window  Help
import pandas as pd

# Create the DataFrame
df = pd.DataFrame()

df['Name'] = ['John', 'Emma', 'Liam', 'Olivia']
df['Age'] = [20, 19, 21, 18]
df['Student'] = [True, True, False, True]

# Create a new row
new_row = pd.DataFrame(
    [['Sophia', 22, False]],
    columns=['Name', 'Age', 'Student']
)

# Add the new row to the DataFrame
df = pd.concat([df, new_row], ignore_index=True)

# Display the updated DataFrame
print(df)
```

Output:

```
     Name  Age  Student
0    John   20     True
1    Emma   19     True
2    Liam   21    False
3   Olivia  18     True
4   Sophia  22    False
>>>
```

Data Analysis with SciPy

1.Importing Required Libraries

Code:

File Edit Format Run Options Window Help

```
import numpy as np
import pandas as pd
from scipy import stats

print("Libraries imported successfully.")
import numpy as np
from scipy import stats

data = [10, 20, 30, 40, 50]

print("Data:", data)
print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Mode:", stats.mode(data, keepdims=True))
```

2.Measures of Central Tendency Using SciPy

Code:

File Edit Format Run Options Window Help

```
import numpy as np
import pandas as pd
from scipy import stats

print("Libraries imported successfully.")
import numpy as np
from scipy import stats

data = [10, 20, 30, 40, 50]

print("Data:", data)
print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Mode:", stats.mode(data, keepdims=True))
```

3.Probability Distribution Analysis Using SciPy

Code:

File Edit Format Run Options Window Help

```
from scipy.stats import norm

probability = norm.cdf(85, loc=70, scale=10)

print("Probability:", round(probability, 4))
```

4.Hypothesis Testing

Code:

File Edit Format Run Options Window Help

```
from scipy import stats

data = [22, 25, 19, 24, 28, 30]

t_stat, p_value = stats.ttest_1samp(data, 25)

print("T-Statistic:", round(t_stat, 3))
print("P-Value:", round(p_value, 3))
```

5.Correlation Analysis

Code:

```
Edit  Format  Run  Options  Window  Help
from scipy.stats import pearsonr

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

corr, p_value = pearsonr(x, y)

print("Correlation:", corr)
print("P-Value:", p_value)
```

6. Linear Algebra Operations

Code:

```
File  Edit  Format  Run  Options  Window  Help
from scipy import linalg

A = [[3, 2],
     [1, 2]]

B = [5, 5]

solution = linalg.solve(A, B)

print("Solution:", solution)
```

7.Optimization Using SciPy

Code:

```
File  Edit  Format  Run  Options  Window  Help
from scipy.optimize import minimize

def objective(x):
    return x**2 + 4

result = minimize(objective, x0=5)

print("Optimal Value of x:", result.x)
print("Minimum Function Value:", result.fun)
```

Output:

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr  9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/addinglibraries.py
Libraries imported successfully.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/addinglibraries.py
Libraries imported successfully.
Data: [10, 20, 30, 40, 50]
Mean: 30.0
Median: 30.0
Mode: ModeResult(mode=array([10]), count=array([1]))
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/pda using scipy.py
Probability: 0.9332
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/hypothesistesting.py
T-Statistic: -0.205
P-Value: 0.846
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/correlational analysis.py
Correlation: 1.0
P-Value: 0.0
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/Linear operation.py
Solution: [0.  2.5]
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/optimization using scipy.py
Optimal Value of x: [-2.62955131e-08]
Minimum Function Value: 4.000000000000001
>>>
```

Scikit Library in Python

1. Logistic Regression (Supervised Learning)

Code:

```
File  Edit  Format  Run  Options  Window  Help
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load Iris dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Standardize the data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train Logistic Regression model
log_reg = LogisticRegression()

log_reg.fit(X_train, y_train)

# Predict test data
y_pred = log_reg.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Output:

```
Accuracy: 1.0
>>>
```

2. K Means Clustering (Unsupervised Learning)

Code:

```
File  Edit  Format  Run  Options  Window  Help
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

# Load Iris dataset
iris = load_iris()

# Create KMeans model
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

# Train the model
kmeans.fit(iris.data)

# Get cluster labels
cluster_labels = kmeans.labels_

print("Cluster Labels:")
print(cluster_labels)
```

Output:

```
Cluster Labels:
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 2 2 2 2 0 2 2 2 2
 2 2 0 0 2 2 2 2 0 2 0 2 0 2 2 0 0 2 2 2 2 2 0 2 2 2 2 0 2 2 2 0 2 2 2 0 2
 2 0]
>>>
```